Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются. За частично верное решение начисляются баллы.

## Задача 1. Оставить цифры не больше порога

**2 балла**

Напишите функцию `keep_digits_at_most(number, threshold)`.

`number` - целое неотрицательное число. `threshold` - целое число от 0 до 9. Оставьте в записи `number` только цифры, которые не больше `threshold`.

Порядок оставшихся цифр должен совпадать с их порядком в исходном числе. Верните результат как целое число: начальные нули в нём не сохраняются. Если ни одной цифры не осталось, верните `0`. Число `0` считайте записью из одной цифры `0`.

Преобразовывать число или отдельные цифры в строки нельзя.

```
def keep_digits_at_most(number: int, threshold: int) -> int:
    ...
```

Примеры:

```
assert keep_digits_at_most(58216, 5) == 521
assert keep_digits_at_most(90802, 2) == 2
assert keep_digits_at_most(987, 3) == 0
assert keep_digits_at_most(1002, 9) == 1002
assert keep_digits_at_most(0, 0) == 0
```

## Задача 2. Следующее слово короче

**2 балла**

Напишите функцию `shrinking_neighbors(text)`.

Верните список пар соседних слов, для которых длина второго слова строго меньше длины первого. Каждая пара должна быть кортежем `(первое слово, второе слово)` в порядке их появления в строке.

Проверяются все соседние пары исходной строки. Одно слово может входить в две пары. Сохраните повторы и исходный регистр букв.

Строка содержит только буквы и пробелы. Пробелов между словами, в начале и конце строки может быть несколько. Если подходящих пар нет или слов меньше двух, верните пустой список.

```
def shrinking_neighbors(text: str) -> list[tuple[str, str]]:
    ...
```

Примеры:

```
assert shrinking_neighbors('далеко дом вижу я') == [('далеко', 'дом'), ('вижу', 'я')]
assert shrinking_neighbors('кот мы я') == [('кот', 'мы'), ('мы', 'я')]
assert shrinking_neighbors('кот кит') == []
assert shrinking_neighbors('  Кот   Кит  ') == []
assert shrinking_neighbors('Привет') == []
assert shrinking_neighbors('   ') == []
assert shrinking_neighbors('') == []
```

## Задача 3. Первый блок положительных чисел

**2,5 балла**

Напишите функцию `first_positive_run(values, k)`.

`values` - список целых чисел, `k` - целое неотрицательное число.

Верните новый список из первого непрерывного блока длины `k`, в котором все числа строго положительные. Если таких блоков несколько, выберите тот, который начинается раньше. Элементы блока должны идти подряд в исходном списке: пропускать неподходящие элементы внутри блока нельзя.

Если такого блока нет, верните пустой список. При `k == 0` также верните пустой список и не просматривайте элементы. Как только первый подходящий блок найден, просмотр исходного списка должен прекратиться.

```
def first_positive_run(values: list[int], k: int) -> list[int]:
    ...
```

Примеры:

```
assert first_positive_run([1, 0, 3, 5, 7], 2) == [3, 5]
assert first_positive_run([1, 3, 5], 2) == [1, 3]
assert first_positive_run([1, -2, 3], 2) == []
assert first_positive_run([-3, 0, 2], 1) == [2]
assert first_positive_run([], 2) == []
assert first_positive_run([1, -1, 0], 0) == []
```

## Задача 4. Выдачи книг без учебников

**3,5 балла**

Напишите функцию `count_book_loans(loans, textbooks)`.

`loans` - список названий книг в журнале выдач, `textbooks` - список названий учебников. Учебники нужно исключить из отчёта.

Верните словарь: ключ - книга, значение - число его вхождений в `loans`, если его нет в `textbooks`. Здесь нужно учитывать все повторения в первом списке. Повторы во втором списке ничего не меняют. Значения, встречающиеся только во втором списке, в результат не включайте.

Оба списка содержат строки. Строки сравниваются точно, с учётом регистра; очищать или преобразовывать их не нужно. Порядок ключей не важен. Если после исключения ничего не осталось, верните пустой словарь.

Требование к эффективности: среднее время работы `O(n + m)`, включая подготовку данных, где `n` и `m` - длины двух входных списков. Считайте, что проверка наличия строки в словаре или множестве, добавление записи в словарь и добавление строки в множество в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def count_book_loans(loans: list[str], textbooks: list[str]) -> dict[str, int]:
    ...
```

Примеры:

```
assert count_book_loans(
    ['Дюна', 'Алгебра', 'Дюна', 'Солярис', 'Алгебра'],
    ['Алгебра'],
) == {'Дюна': 2, 'Солярис': 1}
assert count_book_loans(['Дюна', 'Дюна'], ['Дюна', 'Дюна']) == {}
assert count_book_loans(['Дюна', 'Дюна'], []) == {'Дюна': 2}
assert count_book_loans([], ['нет записей']) == {}
```